

```

    simulator=False)
7. print(f"Running on backend: {backend.name}")

8. qc = QuantumCircuit(2, 2)
9. qc.h(0)
10. qc.cx(0, 1)
11. qc.measure([0, 1], [0, 1])

12. transpiled_circuit = transpile(qc, backend)

13. sampler = Sampler(mode=backend)
14. job = sampler.run([transpiled_circuit], shots=1000)
15. print("Job ID:", job.job_id( ))

16. result = job.result( )
17. counts = result[0].data.c.get_counts( )
18. print(counts)

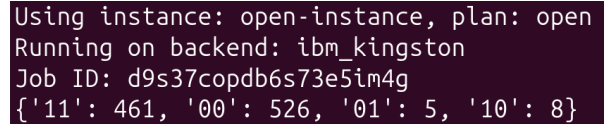
```

The program *qiskit10.py* is so close to *qiskit8.py* that a line-by-line explanation would waste space. Only three things are worth pointing out. First, the quantum part of the program—the four lines (8-11) that create two qubits, apply a Hadamard gate, apply a CNOT gate and measure both qubits—is character for character identical to *qiskit9.py*. Not a single line had to be changed to move from a simulator to a machine cooled to 15 millikelvin. That is a remarkable achievement of the Qiskit software stack, and it is worth pausing to admire it. Second, instead of the preset pass manager used in *qiskit8.py*, we simply call *transpile()* with the chosen backend as an argument. Both rewrite our circuit so that it uses only the operations the target processor physically supports—*transpile()* builds a preset pass manager internally—though it optimises a little harder, defaulting to level 2 where *qiskit8.py* asked for level 1. Third, *save_account()* writes the credentials to the file *~/qiskit/qiskit-ibm.json*, and *set_as_default=True* marks this account as the one to use when more than one has been saved. This is a one-time step: later programs can create a *QiskitRuntimeService()* object with no arguments and simply find the account.

A word of caution. Your API key is a password, not a decorative string. Never publish it, never commit it to a public repository, and never leave it in a screenshot. If one is ever exposed, revoke it on the IBM Quantum Platform and generate a new one.

Before executing the program, activate the virtual environment with the command *conda activate qiskit_env* and make sure that the IBM Runtime package is installed, using *pip install qiskit-ibm-runtime*, exactly as we did in the previous article. Then replace the placeholder “PasteYourAPIKeyHere” with your own API key and run the command *python qiskit10.py*. The output obtained is

shown in Figure 1.



```

Using instance: open-instance, plan: open
Running on backend: ibm_kingston
Job ID: d9s37copdb6s73e5im4g
{'11': 461, '00': 526, '01': 5, '10': 8}

```

Figure 1: Terminal output obtained after executing *qiskit10.py* on an IBM quantum processor

This time the least busy processor turned out to be *ibm_kingston*, a 156-qubit machine belonging to IBM’s Heron R2 family. The measurement statistics returned by the hardware are $\{‘11’: 461, ‘00’: 526, ‘01’: 5, ‘10’: 8\}$. Compare this with the ideal simulator output of the previous article, which contained nothing but 00 and 11. Here, 987 of the 1000 shots behaved themselves and produced the perfectly correlated outcomes we expect from the entangled state $(|00\rangle + |11\rangle)/\sqrt{2}$. The remaining 13 shots produced 01 or 10—outcomes that a perfectly entangled pair is simply not allowed to produce. Those 13 stray counts are the most interesting numbers in this article. On an ideal simulator, entanglement is a promise that is always kept; in this particular run, it was kept 98.7 per cent of the time. Let us now find out who broke the promise.

Where do the errors come from?

In the previous article we introduced the term Noisy Intermediate-Scale Quantum (NISQ) and noted that today’s processors are noisy. That is a little like saying that the monsoon is wet: true, but not very informative. A superconducting quantum computer fails in several distinct ways, each with its own physical origin, its own characteristic magnitude, and its own set of countermeasures. Let us look at the most important ones.

Energy relaxation, usually denoted T1, is the tendency of an excited qubit to give up its energy to the environment and fall back from $|1\rangle$ to $|0\rangle$. A qubit does not stay excited out of politeness; it stays excited only until the surrounding world persuades it otherwise. Both processors we have used—*ibm_marrakesh* in the previous article and *ibm_kingston* in this one—belong to IBM’s Heron family, where T1 is typically 100 to 300 microseconds, though the figure varies from qubit to qubit and from one calibration to the next. Notice that this error is not symmetric: a qubit decays from $|1\rangle$ to $|0\rangle$, but essentially never the other way round, because the refrigerator is far colder than the energy gap between the two levels.

Dephasing, denoted T2, is more subtle and, for readers of the third article in this series, more interesting. Here the qubit does not lose its energy; it loses its phase. Recall from the third article that a superposition carries a relative phase between $|0\rangle$ and $|1\rangle$, and that this phase, though invisible to a direct measurement, decides the outcome of every subsequent operation. Random fluctuations in the qubit’s frequency cause that phase to drift